



Laurea Triennale in informatica-Università di Salerno
Corso di Ingegneria del Software- Prof. Andrea De Lucia



Test Plan CardioCare360

Riferimento	2026_TP_CardioCare360_V1.0
Versione	1.0
Data	04/05/2026
Destinatario	Prof. Andrea De Lucia
Presentato da	Ciro Cannello
Approvato da	/

Indice generale

Revision History.....	3
Team Members.....	3
1. Introduzione.....	4
2. Relazione con altri documenti.....	5
3. Panoramica del sistema.....	6
4. Funzionalità da testare / da non testare.....	7
5. Criteri di successo e fallimento (Pass/Fail).....	9
6. Approccio di testing.....	11
6.1 System Testing.....	11
6.2 Functional Testing.....	12
6.3 Performance Testing.....	14
6.4 Pilot Testing.....	16
6.5 Acceptance Testing.....	18
6.6 Installation Testing.....	20
7. Sospensione e Ripristino dei Test.....	22
8. Materiale e Ambiente di Testing.....	24
8.3 Ambiente hardware.....	25
9. Test Cases.....	27
9.1 Gestione Registrazione.....	27
9.1.1 Registrazione Paziente.....	27
9.1.2 Registrazione Medico.....	27
9.2 Gestione Autenticazione.....	27
9.2.1 Login / Logout.....	27
9.3 Gestione Prenotazioni Visite.....	27
9.3.1 Prenotazione Visita Cardiologica.....	27
9.3.2 Modifica / Annullamento Prenotazione.....	28
9.4 Gestione Esami e Referti.....	28
9.4.1 Inserimento Referto.....	28
9.4.2 Visualizzazione Esame.....	28
9.5 Gestione Terapie.....	28
9.5.1 Aggiunta / Aggiornamento Terapia.....	28
9.6 Gestione Profilo Utente.....	28
9.6.1 Modifica Dati Personali.....	28
9.6.2 Visualizzazione Storico.....	29

Revision History

Data	Versione	Descrizione	Autori
01/05/2026	0.1	Prima stesura del documento	Ciro Cannello
01/05/2026	0.2	Relazione con altri documenti	Ciro Cannello
02/05/2026	0.3	Panoramica del Sistema	Ciro Cannello
02/05/2026	0.4	Feature da Testare non Testare	Ciro Cannello
02/05/2026	0.5	Test Cases	Ciro Cannello
03/05/2026	1.0	Revisione e consegna	Ciro Cannello

Team Members

Ruolo	Nome e Cognome	Acronimo	Email
PM	Ciro Cannello	CC	c.cannello@studenti.unisa.it
TM	Ciro Cannello	CC	c.cannello@studenti.unisa.it

1. Introduzione

Il presente documento descrive il **Piano di Test (Test Plan)** del progetto *CardioCare360*, sviluppato nell'ambito del corso di **Ingegneria del Software** presso l'Università di Salerno. L'obiettivo del piano è definire in modo sistematico le attività di verifica e validazione necessarie per garantire che il sistema soddisfi i requisiti funzionali e non funzionali specificati nel **RAD** (*Requirements Analysis Document*) e nel **SDD** (*System Design Document*).

Il documento stabilisce:

- le **funzionalità da testare** e quelle escluse dal processo di verifica;
- i **criteri di successo e fallimento** dei test;
- l'**approccio metodologico** adottato per le diverse tipologie di test (unit, integration, system, acceptance);
- le **risorse, strumenti e ambienti** utilizzati;
- la **pianificazione e tracciabilità** dei casi di test rispetto ai requisiti del sistema.

Il piano di test è stato redatto con l'obiettivo di assicurare la **qualità complessiva del software**, ridurre il rischio di malfunzionamenti e garantire la conformità del prodotto finale agli standard di progetto e alle aspettative dell'utente finale.

Nota metodologica

Il documento segue la struttura standard IEEE 829, adattata al contesto accademico del progetto *CardioCare360*. Tutte le sezioni successive (da 2 a 11) dettagliano le strategie di test, i casi di prova e i risultati attesi per ciascun modulo del sistema.

2. Relazione con altri documenti

Il presente *Test Plan* è strettamente collegato ai principali documenti di analisi e progettazione del progetto **CardioCare360**, dai quali derivano i requisiti, le funzionalità e le specifiche tecniche oggetto delle attività di verifica e validazione. La tracciabilità tra i documenti consente di garantire coerenza, completezza e correttezza del processo di sviluppo software.

I documenti di riferimento sono:

- **RAD – Requirements Analysis Document** Contiene la definizione dei requisiti funzionali e non funzionali, i casi d'uso, gli scenari brevi e le regole di business. Il Test Plan utilizza il RAD come base per la definizione delle funzionalità da testare e per la costruzione dei casi di test (Test Cases), assicurando la copertura di tutti i requisiti identificati.
- **SDD – System Design Document** Descrive l'architettura del sistema, i moduli software, le interfacce, i diagrammi UML e le scelte progettuali. Il Test Plan fa riferimento al SDD per definire le tipologie di test (unit, integration, system) e per verificare la corretta implementazione dei componenti progettati.
- **ODD – Object Design Document** Specifica il comportamento operativo del sistema, i flussi principali, le interazioni tra attori e le procedure di utilizzo. Il Test Plan utilizza l'ODD per validare la coerenza dei flussi operativi e per costruire test end-to-end basati sui comportamenti reali del sistema.
- **Documentazione di progetto e repository di sviluppo** Comprende il codice sorgente, le specifiche tecniche dei moduli, le API e le configurazioni dell'ambiente di sviluppo. Questi elementi sono utilizzati per definire i test tecnici, le condizioni di esecuzione e gli strumenti necessari.

La relazione tra i documenti garantisce che il processo di testing sia allineato alle fasi di analisi, progettazione e implementazione, assicurando una verifica completa e affidabile del sistema CardioCare360.

3. Panoramica del sistema

Il sistema **CardioCare360** è una piattaforma web dedicata alla **gestione integrata delle attività cardiologiche** tra pazienti, medici e amministratori. L'obiettivo principale è fornire un ambiente digitale sicuro e intuitivo per la **prenotazione delle visite**, la **gestione degli esami**, la **consultazione dei referti** e il **monitoraggio delle terapie**.

Il sistema è stato progettato secondo un'architettura **multi-layer** basata su **Spring Boot** per il backend e **React** per il frontend, con un database relazionale che garantisce la persistenza e la coerenza dei dati. Ogni modulo è stato definito in modo da riflettere i flussi operativi descritti negli **Activity Diagram** e nei **Use Case** del RAD.

Componenti principali

- **Modulo Utente** Gestisce la registrazione, l'autenticazione e la profilazione di pazienti e medici. Include la protezione delle pagine riservate e la gestione delle sessioni.
- **Modulo Prenotazioni** Permette al paziente di selezionare il tipo di visita, la data, l'orario e il medico disponibile. Il sistema mostra solo le disponibilità coerenti con la visita scelta e salva ogni step fino alla conferma finale.
- **Modulo Esami e Referti** Consente al medico di inserire referti e al paziente di visualizzare gli esami completati. Ogni referto è associato al paziente e all'esame corrispondente.
- **Modulo Terapie** Gestisce l'inserimento, l'aggiornamento e la consultazione delle terapie prescritte. Supporta la tracciabilità delle modifiche e la verifica della coerenza con gli esami precedenti.
- **Modulo Amministrazione** Permette la supervisione del sistema, la gestione degli utenti e il monitoraggio delle attività globali.

Obiettivi di qualità

Il sistema mira a garantire:

- **Affidabilità** → tutti i dati clinici sono gestiti in modo sicuro e persistente.
- **Usabilità** → interfacce chiare e flussi guidati per ogni tipo di utente.
- **Scalabilità** → architettura modulare e facilmente estendibile.
- **Manutenibilità** → codice strutturato secondo pattern MVC e separazione dei livelli logici.
- **Sicurezza** → autenticazione e autorizzazione basate su ruoli, con protezione delle operazioni sensibili.

Sintesi operativa

Il *Test Plan* si focalizza sulla verifica di ciascun modulo e dei flussi principali:

- Registrazione e autenticazione utenti
- Prenotazione e gestione visite
- Inserimento e consultazione referti
- Gestione terapie e profilo utente

Ogni test case è progettato per validare la **correttezza funzionale**, la **robustezza** e la **coerenza dei dati** all'interno del sistema CardioCare360.

4. Funzionalità da testare / da non testare

La presente sezione identifica le funzionalità del sistema CardioCare360 oggetto di verifica tramite test automatizzati (unit test JUnit/Mockito), nonché quelle escluse dal perimetro di questa fase di test. Lo scope è stato definito in modo da garantire piena coerenza tra quanto pianificato in questo documento, i casi di test specificati nel TCS, e i test realmente implementati ed eseguiti sul codice sorgente.

Funzionalità da testare

Le seguenti funzionalità sono state sottoposte a test automatizzati a livello di logica applicativa (Service):

- Gestione Parametri Vitali
 - Inserimento parametro e verifica soglie (valore nella norma, ai limiti esatti, fuori soglia, nessuna soglia configurata)
- Gestione Registrazione
 - Registrazione paziente con dati validi
 - Gestione errori (email duplicata, codice fiscale duplicato)
- Gestione Autenticazione
 - Login con credenziali corrette
 - Login con credenziali errate (utente non trovato, password errata)
- Gestione Appuntamenti
 - Creazione con dati validi
 - Creazione con data nel passato (validazione)
 - Eliminazione da parte del proprietario
 - Eliminazione negata a utente non autorizzato
- Gestione Terapie
 - Assegnazione terapia con dati validi
 - Regola anti-duplicato (un appuntamento non può avere più di una terapia)
 - Gestione errore paziente non trovato
- Gestione Esami
 - Aggiornamento dello stato di un esame
 - Eliminazione di un esame (esistente e inesistente)

- Recupero del referto associato a un esame
- Gestione Disponibilità Medico
- Creazione di una fascia oraria con controllo anti-duplicato
- Modifica di una disponibilità esistente
- Consultazione delle disponibilità per giorno

Livelli di test applicati

Coerentemente con la richiesta del Problem Statement di un “Piano di test di sistema”, la suite di test copre tre livelli, applicati a tutti i moduli sopra elencati:

- Test di Unità (27) — JUnit 5 e Mockito: verificano la logica dei Service isolata, con Repository sostituiti da mock.
- Test di Integrazione (4) — MockMvc e database H2 in memoria: verificano Controller, Service e Repository realmente collegati tra loro, sul modulo Appuntamenti e Autenticazione.
- Test di Sistema (4) — TestRestTemplate su applicazione realmente avviata (porta reale): verificano il flusso end-to-end sul modulo Parametri Vitali (generazione alert) e sulle regole di autorizzazione per ruolo.
- Test State-based (5) — costruzione della macchina a stati finiti dell’entità Appuntamento e verifica dei cammini di transizione, comprese transizioni non previste dallo statechart del RAD.
- Test Category-Partition (3) — combinazioni sistematiche delle categorie di input (esistenza di Medico, Farmaco, Appuntamento) sul metodo creaTerapia().

Totale complessivo: 43 test automatizzati.

Funzionalità da non testare

Le seguenti funzionalità sono escluse dal perimetro di questa fase di test automatizzato:

- Funzionalità non implementate nella versione corrente del sistema
- Aggiornamento di una terapia già assegnata
- Recupero password
- Gestione dinamica delle soglie cliniche da parte del medico
- Notifiche via SMS, integrazione con sistemi sanitari esterni, pagamenti online
- Funzionalità pianificate ma non ancora automatizzate in questa fase
- Registrazione medico da parte dell’amministratore
- Logout e invalidazione sessione
- Protezione delle pagine riservate (autorizzazione a livello di route)
- Flusso di prenotazione visita tramite interfaccia (selezione tipo/data/orario/medico)
- Modifica di un appuntamento esistente

- Caricamento referto da parte del medico e visualizzazione/download da parte del paziente
- Modifica dati personali e gestione profilo
- Messaggistica/conversazioni e notifiche
- Operazioni di amministrazione su medici e pazienti

Queste funzionalità potranno essere oggetto di future estensioni della suite di test (unitari, di integrazione o manuali), ma non rientrano nel perimetro dei test automatizzati realizzati in questa fase del progetto.

- Funzionalità non critiche o dipendenti da servizi esterni
- Personalizzazione grafica, animazioni e transizioni dell'interfaccia
- Invio email reale (simulato per scopi accademici)

Le funzionalità escluse non compromettono la qualità complessiva del sistema, non influenzano la logica di business critica già verificata, e non sono richieste ai fini della valutazione accademica in questa fase.

5. Criteri di successo e fallimento (Pass/Fail)

I criteri di successo e fallimento definiscono le condizioni che determinano l'esito dei test eseguiti sul sistema **CardioCare360**. Essi garantiscono una valutazione oggettiva e misurabile delle funzionalità, assicurando che ogni componente rispetti i requisiti specificati nel **RAD**, nel **SDD** e nei flussi operativi del sistema.

Criteri di successo (Pass)

Un test è considerato **superato (Pass)** quando:

- il comportamento del sistema **corrisponde esattamente** ai requisiti funzionali previsti;
- l'output generato è **corretto, completo e coerente** con i dati presenti nel database;
- non si verificano errori, eccezioni o comportamenti inattesi;
- le interfacce mostrano informazioni corrette e aggiornate;
- le operazioni critiche (registrazione, login, prenotazione, inserimento referti, gestione terapie) vengono completate **senza interruzioni**;
- il sistema rispetta i vincoli di sicurezza, autorizzazione e protezione delle pagine riservate;
- il flusso operativo segue la sequenza definita negli Activity Diagram del progetto;
- eventuali messaggi di errore sono **chiari, contestuali e conformi** alle specifiche.

Un test è considerato "Pass" anche quando il sistema gestisce correttamente condizioni limite o input non validi, mostrando messaggi appropriati e impedendo operazioni non consentite.

Criteri di fallimento (Fail)

Un test è considerato **fallito (Fail)** quando si verifica almeno una delle seguenti condizioni:

- il sistema produce un output **errato, incompleto o incoerente** rispetto ai requisiti;

- si verificano errori di runtime, eccezioni o crash dell'applicazione;
- il flusso operativo non segue la sequenza prevista (salti di step, dati non salvati, comportamenti anomali);
- vengono mostrati dati non aggiornati, non pertinenti o appartenenti ad altri utenti;
- le operazioni critiche non vengono completate (prenotazione non salvata, referto non registrato, terapia non aggiornata);
- il sistema permette azioni non autorizzate (accesso a pagine riservate senza login, modifica dati di altri utenti);
- i messaggi di errore sono assenti, fuorvianti o non conformi alle specifiche;
- il sistema non rispetta i vincoli di sicurezza o integrità dei dati.

Un test è “Fail” anche quando il comportamento osservato è ambiguo, non deterministico o non replicabile in successive esecuzioni.

Criteri di accettazione del sistema

Il sistema **CardioCare360** può essere considerato idoneo alla fase di consegna quando:

- **tutti i test critici** (registrazione, autenticazione, prenotazioni, referti, terapie) risultano “Pass”;
- almeno il **95% dei test funzionali** risulta superato;
- non sono presenti bug bloccanti o ad alta priorità;
- i bug minori non compromettono i flussi principali e possono essere risolti in release successive;
- il sistema rispetta i requisiti di qualità definiti nel RAD e nel SDD.

6. Approccio di testing

L'approccio di testing adottato per il progetto **CardioCare360** segue una strategia strutturata e progressiva, basata sui principi della verifica e validazione del software. L'obiettivo è garantire che ogni componente del sistema soddisfi i requisiti funzionali e non funzionali definiti nel **RAD**, rispetti le specifiche architetturali del **SDD** e sia coerente con i flussi operativi descritti negli **Activity Diagram**.

Il processo di testing è organizzato in livelli, ciascuno focalizzato su aspetti specifici del sistema, dalla verifica dei singoli moduli fino alla validazione dell'intero flusso applicativo.

6.1 System Testing

Il **System Testing** rappresenta la fase in cui il sistema **CardioCare360** viene testato nella sua interezza, come prodotto completo e integrato. L'obiettivo è verificare che tutte le componenti — backend, frontend e database — lavorino in modo coerente, garantendo il corretto funzionamento dei flussi operativi principali.

Obiettivi

- Validare la conformità del sistema ai requisiti funzionali e non funzionali definiti nel **RAD**.
- Verificare la corretta integrazione tra i moduli (Utente, Prenotazioni, Esami, Terapie, Profilo).
- Accertare la stabilità e la coerenza dei dati durante l'esecuzione dei flussi end-to-end.
- Controllare la gestione degli errori e delle eccezioni in condizioni normali e limite.
- Garantire la sicurezza e la protezione delle operazioni riservate.

Ambito di test

Il System Testing copre i seguenti flussi principali:

- Registrazione e autenticazione utenti.
- Prenotazione, modifica e annullamento visite.
- Inserimento e visualizzazione referti.
- Gestione terapie e profilo utente.
- Accesso e navigazione tra le sezioni del sistema.

Ogni flusso viene testato in condizioni reali, simulando l'interazione di pazienti e medici.

Criteri di successo

Un test di sistema è considerato **Pass** quando:

- Tutte le funzionalità principali producono risultati corretti e coerenti.
- Non si verificano errori bloccanti o crash.
- I dati inseriti e recuperati dal database sono consistenti.
- Le interfacce rispondono correttamente alle azioni dell'utente.

- I messaggi di errore sono chiari e contestuali.

Strumenti utilizzati

- **Postman** → verifica API REST.
- **Browser Developer Tools** → analisi frontend e flussi di navigazione.
- **Database integrato in Spring Boot** → usato per testare la persistenza dei dati e la coerenza delle operazioni CRUD.
- **LibreOffice Calc / Writer** → per la documentazione dei risultati dei test e la tracciabilità dei casi di prova.

Risultati attesi

Il System Testing deve confermare che **CardioCare360** sia stabile, coerente e pronto per le fasi successive di **Acceptance Testing** e **Installation Testing**.

6.2 Functional Testing

Il **Functional Testing** ha l'obiettivo di verificare che ogni funzionalità del sistema **CardioCare360** rispetti i requisiti definiti nel **RAD** e sia coerente con i flussi operativi descritti negli **Activity Diagram**. Questo tipo di testing si concentra esclusivamente sul comportamento osservabile del sistema, indipendentemente dall'implementazione interna.

Obiettivi

- Validare che ogni funzionalità produca risultati corretti e coerenti.
- Verificare che le interazioni utente-sistema seguano i flussi previsti.
- Assicurare che i dati inseriti, modificati o visualizzati siano corretti e aggiornati.
- Controllare la gestione degli errori e dei messaggi informativi.
- Garantire che il sistema impedisca operazioni non consentite o non autorizzate.

Ambito del Functional Testing

Il Functional Testing copre tutte le funzionalità principali del sistema, tra cui:

- **Registrazione paziente e medico**
- **Login / Logout e protezione delle pagine riservate**
- **Prenotazione visita cardiologica**
- **Modifica e annullamento prenotazione**
- **Inserimento referto da parte del medico**
- **Visualizzazione esame da parte del paziente**
- **Aggiunta e aggiornamento terapia**
- **Modifica dati personali del profilo utente**

- **Visualizzazione storico visite/esami**

Ogni funzionalità viene testata con input validi, input non validi e condizioni limite.

Tecniche utilizzate

- **Black-box testing** Utilizzato per verificare il comportamento del sistema senza analizzare il codice. Ideale per testare flussi come prenotazioni, autenticazione, referti, terapie.
- **Testing basato sui requisiti** Ogni test case è tracciato a uno o più requisiti del RAD.
- **Testing basato sui flussi operativi** I test seguono gli Activity Diagram del progetto per garantire coerenza operativa.

Strumenti utilizzati

- **Postman** → verifica delle API REST e delle chiamate tra frontend e backend.
- **Browser Developer Tools** → analisi del comportamento del frontend, richieste HTTP e flussi di navigazione.
- **Database integrato in Spring Boot** → verifica della persistenza dei dati e delle operazioni CRUD.
- **LibreOffice Writer / Calc** → documentazione dei risultati dei test e tracciabilità dei casi di prova.

Criteri di successo

Un test funzionale è considerato **Pass** quando:

- il comportamento del sistema coincide con quanto previsto dal RAD;
- i dati inseriti, modificati o visualizzati sono corretti e coerenti;
- non si verificano errori o comportamenti inattesi;
- i messaggi di errore sono chiari e contestuali;
- il flusso operativo segue la sequenza prevista.

Risultati attesi

Il Functional Testing deve confermare che ogni funzionalità del sistema **CardioCare360** sia corretta, stabile e conforme ai requisiti, garantendo la qualità complessiva del prodotto prima delle fasi successive di **Performance, Pilot, Acceptance e Installation Testing**.

6.3 Performance Testing

Il **Performance Testing** ha l'obiettivo di valutare la capacità del sistema **CardioCare360** di rispondere in modo rapido, stabile e coerente durante l'esecuzione delle operazioni principali. Questa fase di testing verifica che il sistema mantenga un comportamento adeguato anche in condizioni di carico moderato, garantendo un'esperienza fluida per pazienti e medici.

Obiettivi

- Misurare i tempi di risposta delle API principali (registrazione, login, prenotazioni, referti, terapie).
- Verificare la stabilità del sistema durante l'esecuzione di operazioni ripetute.
- Assicurare che il sistema gestisca correttamente richieste multiple in sequenza.
- Controllare che il database mantenga coerenza e integrità dei dati anche dopo operazioni frequenti.
- Identificare eventuali rallentamenti o punti critici nei flussi operativi.

Ambito del Performance Testing

Il Performance Testing si concentra sui flussi più utilizzati del sistema:

- **Autenticazione (Login / Logout)**
- **Prenotazione visita cardiologica**
- **Modifica / annullamento prenotazione**
- **Inserimento referto**
- **Visualizzazione esame**
- **Aggiornamento terapia**
- **Consultazione storico visite/esami**

Ogni flusso viene eseguito più volte per misurare la stabilità e la costanza dei tempi di risposta.

Tecniche utilizzate

- **Testing manuale ripetuto** Esecuzione consecutiva delle stesse operazioni per rilevare eventuali rallentamenti.
- **Testing basato sui flussi operativi** Le operazioni vengono eseguite seguendo gli Activity Diagram del RAD per garantire coerenza.
- **Testing basato sui dati** Verifica della persistenza e integrità dei dati dopo operazioni ripetute.

Strumenti utilizzati

- **Postman** → misurazione dei tempi di risposta delle API REST.
- **Browser Developer Tools** → analisi delle prestazioni del frontend (Network, Timeline).

- **Database integrato in Spring Boot** → verifica della persistenza dei dati dopo operazioni ripetute.
- **LibreOffice Writer / Calc** → registrazione dei risultati e confronto dei tempi di risposta.

Criteri di successo

Il Performance Testing è considerato **Pass** quando:

- i tempi di risposta delle API rimangono entro valori accettabili per un'applicazione web accademica;
- il sistema non mostra rallentamenti significativi durante operazioni ripetute;
- non si verificano crash, blocchi o errori di timeout;
- i dati rimangono coerenti e correttamente salvati nel database;
- il frontend risponde in modo fluido alle interazioni dell'utente.

Risultati attesi

Il Performance Testing deve confermare che **CardioCare360** è in grado di gestire le operazioni principali con tempi di risposta adeguati, mantenendo stabilità e coerenza dei dati. Questa fase prepara il sistema alle successive attività di **Pilot Testing**, **Acceptance Testing** e **Installation Testing**.

6.4 Pilot Testing

Il **Pilot Testing** rappresenta una fase intermedia del processo di verifica del sistema

CardioCare360, in cui un gruppo ristretto di utenti (pazienti e medici simulati) esegue i flussi principali dell'applicazione in un ambiente controllato. L'obiettivo è valutare il comportamento del sistema in condizioni d'uso reale, identificando eventuali problemi di usabilità, coerenza dei dati, navigazione o comprensione delle funzionalità.

Questa fase permette di raccogliere feedback preliminari prima dell'Acceptance Testing, riducendo il rischio di errori nelle fasi finali e migliorando la qualità complessiva del sistema.

Obiettivi

- Verificare che i flussi operativi principali siano intuitivi e facilmente eseguibili dagli utenti.
- Valutare la chiarezza delle interfacce e dei messaggi informativi.
- Identificare eventuali difficoltà di navigazione o passaggi non chiari.
- Controllare la coerenza dei dati durante l'esecuzione dei flussi end-to-end.
- Raccogliere feedback qualitativo sull'esperienza d'uso del sistema.
- Individuare eventuali anomalie non rilevate durante Functional e System Testing.

Ambito del Pilot Testing

Il Pilot Testing copre i flussi principali del sistema, simulando l'esperienza reale di pazienti e medici:

- Registrazione utente (paziente e medico).
- Login e accesso alle sezioni riservate.
- Prenotazione visita cardiologica.
- Modifica e annullamento prenotazione.
- Inserimento referto da parte del medico.
- Visualizzazione esame da parte del paziente.
- Aggiunta e aggiornamento terapia.
- Consultazione dello storico visite/esami.
- Navigazione generale tra le pagine del sistema.

Ogni flusso viene eseguito seguendo gli Activity Diagram del RAD per garantire coerenza con il comportamento previsto.

Tecniche utilizzate

- **Black-box testing** Gli utenti simulati interagiscono con il sistema senza conoscere la logica interna, valutando esclusivamente il comportamento osservabile.
- **Testing basato sui flussi operativi** I test seguono i flussi descritti negli Activity Diagram, verificando che ogni passaggio sia chiaro e correttamente implementato.

- **Testing basato sull'esperienza utente (UX)** Valutazione della facilità d'uso, della chiarezza delle interfacce e della comprensibilità dei messaggi.
- **Testing manuale guidato** Gli utenti eseguono scenari predefiniti per verificare la correttezza dei flussi principali.

Strumenti utilizzati

- **Browser Web** → esecuzione dei flussi e valutazione dell'interfaccia.
- **Browser Developer Tools** → analisi delle richieste HTTP e verifica del comportamento del frontend.
- **Postman** → controllo delle API in caso di anomalie rilevate durante il pilot.
- **LibreOffice Writer / Calc** → raccolta feedback, registrazione dei risultati e documentazione delle osservazioni.

Criteri di successo

Il Pilot Testing è considerato **Pass** quando:

- gli utenti riescono a completare i flussi principali senza difficoltà significative;
- le interfacce risultano chiare e comprensibili;
- i messaggi informativi e di errore sono coerenti e contestuali;
- non emergono problemi di navigazione o passaggi confusi;
- i dati risultano correttamente salvati e visualizzati;
- non si verificano crash, blocchi o comportamenti anomali.

Risultati attesi

Il Pilot Testing deve confermare che **CardioCare360** è pronto per essere sottoposto all'**Acceptance Testing**, garantendo che l'esperienza utente sia fluida, coerente e priva di ostacoli. Questa fase permette di individuare eventuali miglioramenti prima della validazione finale del sistema.

6.5 Acceptance Testing

L'**Acceptance Testing** rappresenta la fase finale del processo di verifica del sistema **CardioCare360**. In questa fase il sistema viene valutato dal punto di vista dell'utente finale (paziente, medico e amministratore simulati), con l'obiettivo di confermare che tutte le funzionalità implementate soddisfino i requisiti definiti nel **RAD** e siano coerenti con i flussi operativi descritti negli **Activity Diagram**.

L'Acceptance Testing costituisce il momento decisivo per stabilire se il sistema è pronto per la consegna e per l'utilizzo reale. Questa fase permette di validare non solo la correttezza tecnica, ma anche l'adeguatezza funzionale e l'esperienza d'uso complessiva.

Obiettivi

- Verificare che il sistema soddisfi tutti i requisiti funzionali e non funzionali previsti nel RAD.
- Confermare che i flussi operativi principali siano completi, corretti e coerenti con quanto progettato.
- Validare la chiarezza delle interfacce e la comprensibilità dei messaggi informativi.
- Assicurare che il sistema sia utilizzabile senza difficoltà da parte degli utenti finali.
- Identificare eventuali anomalie residue prima della consegna definitiva.
- Stabilire formalmente l'idoneità del sistema alla fase di rilascio.

Ambito dell'Acceptance Testing

L'Acceptance Testing copre tutte le funzionalità principali del sistema, includendo:

- Registrazione paziente e medico.
- Login, logout e protezione delle pagine riservate.
- Prenotazione visita cardiologica.
- Modifica e annullamento prenotazione.
- Inserimento referto da parte del medico.
- Visualizzazione esame da parte del paziente.
- Aggiunta e aggiornamento terapia.
- Consultazione dello storico visite/esami.
- Navigazione generale tra le sezioni del sistema.
- Verifica della coerenza dei dati tra i vari moduli.

Ogni funzionalità viene testata seguendo gli scenari descritti nel RAD e gli Activity Diagram del progetto.

Tecniche utilizzate

- **Black-box testing** Gli utenti simulati verificano il comportamento del sistema senza analizzare la logica interna.

- **Testing basato sui requisiti** Ogni test case è tracciato a uno o più requisiti del RAD, garantendo copertura completa.
- **Testing basato sui flussi operativi** I test seguono gli Activity Diagram per assicurare coerenza con il comportamento previsto.
- **Testing manuale guidato** Gli utenti eseguono scenari predefiniti per validare la correttezza dei flussi principali.
- **Testing basato sull'esperienza utente (UX)** Valutazione della facilità d'uso, della chiarezza delle interfacce e della comprensibilità dei messaggi.

Strumenti utilizzati

- **Browser Web** → esecuzione dei flussi e valutazione dell'interfaccia.
- **Browser Developer Tools** → analisi delle richieste HTTP e verifica del comportamento del frontend.
- **Postman** → controllo delle API in caso di anomalie rilevate durante l'Acceptance Testing.
- **LibreOffice Writer / Calc** → documentazione dei risultati, raccolta feedback e tracciabilità dei test.

Criteri di successo

L'Acceptance Testing è considerato **Pass** quando:

- tutte le funzionalità principali risultano corrette e conformi ai requisiti del RAD;
- gli utenti riescono a completare i flussi operativi senza difficoltà;
- le interfacce risultano chiare e coerenti con le aspettative;
- i messaggi informativi e di errore sono corretti e contestuali;
- non emergono problemi di navigazione o incoerenze nei dati;
- non si verificano crash, blocchi o comportamenti anomali;
- il sistema risulta complessivamente pronto per la consegna.

Risultati attesi

L'Acceptance Testing deve confermare che **CardioCare360** è conforme ai requisiti, stabile, usabile e pronto per il rilascio. Questa fase rappresenta la validazione finale del sistema e costituisce il punto di approvazione per la consegna del progetto.

6.6 Installation Testing

L'**Installation Testing** ha l'obiettivo di verificare che il sistema **CardioCare360** possa essere installato, configurato e avviato correttamente nell'ambiente previsto. Questa fase assicura che tutti i componenti software — backend, frontend e database — siano correttamente predisposti e funzionanti dopo la procedura di installazione, garantendo che il sistema sia pronto per l'utilizzo da parte degli utenti finali.

L'Installation Testing è fondamentale per confermare che il sistema possa essere distribuito senza errori, che le dipendenze siano correttamente risolte e che l'ambiente di esecuzione sia configurato in modo coerente con quanto definito nel **SDD**.

Obiettivi

- Verificare che l'installazione del backend avvenga senza errori.
- Assicurare che il frontend venga avviato correttamente e comunichi con il backend.
- Controllare che il database sia correttamente configurato e accessibile.
- Validare che tutte le dipendenze software siano presenti e funzionanti.
- Confermare che il sistema possa essere avviato e utilizzato subito dopo l'installazione.
- Identificare eventuali problemi legati alla configurazione dell'ambiente.

Ambito dell'Installation Testing

L'Installation Testing copre le seguenti attività:

- Configurazione dell'ambiente di sviluppo (Java, Node.js, Spring Boot, React).
- Installazione delle dipendenze backend tramite Maven.
- Installazione delle dipendenze frontend tramite npm.
- Avvio del backend e verifica della corretta esposizione delle API.
- Avvio del frontend e verifica della corretta comunicazione con il backend.
- Configurazione del database integrato in Spring Boot.
- Verifica della creazione automatica delle tabelle e della persistenza dei dati.
- Controllo della corretta visualizzazione delle pagine principali del sistema.

Ogni attività viene eseguita seguendo le procedure descritte nel SDD e nella documentazione tecnica del progetto.

Tecniche utilizzate

- **Testing manuale guidato** L'installazione viene eseguita passo per passo, verificando ogni componente.
- **Testing basato sulla configurazione** Controllo della correttezza dei file di configurazione (application.properties, package.json, ecc.).
- **Testing basato sull'avvio del sistema** Verifica che backend e frontend si avviino correttamente e siano accessibili.

- **Testing basato sulla connettività** Controllo della comunicazione tra frontend, backend e database.

Strumenti utilizzati

- **Terminale / Prompt dei comandi** → esecuzione dei comandi di installazione e avvio.
- **Browser Web** → verifica dell'interfaccia e delle pagine principali.
- **Browser Developer Tools** → analisi delle richieste HTTP e verifica della comunicazione con il backend.
- **Postman** → controllo delle API dopo l'avvio del backend.
- **LibreOffice Writer / Calc** → documentazione dei risultati e tracciabilità delle operazioni.

Criteri di successo

L'Installation Testing è considerato **Pass** quando:

- backend e frontend vengono installati senza errori;
- tutte le dipendenze risultano correttamente installate;
- il backend espone correttamente le API;
- il frontend comunica correttamente con il backend;
- il database viene inizializzato correttamente e risponde alle operazioni CRUD;
- il sistema si avvia senza crash o errori bloccanti;
- le pagine principali risultano accessibili e funzionanti.

Risultati attesi

L'Installation Testing deve confermare che **CardioCare360** può essere installato e avviato correttamente nell'ambiente previsto, garantendo che il sistema sia pronto per la distribuzione e l'utilizzo reale. Questa fase conclude il ciclo di testing e prepara il sistema alla consegna finale.

7. Sospensione e Ripristino dei Test

La sezione dedicata alla **sospensione e ripristino dei test** definisce le condizioni in cui le attività di testing del sistema **CardioCare360** devono essere temporaneamente interrotte e le procedure da seguire per riprendere correttamente le operazioni. Questa parte del Test Plan è essenziale per garantire continuità, tracciabilità e controllo del processo di verifica, evitando che anomalie, problemi tecnici o condizioni impreviste compromettano la qualità complessiva del sistema.

Condizioni di sospensione dei test

Le attività di testing devono essere sospese quando si verifica almeno una delle seguenti condizioni:

- **Bug bloccanti (Severity: High)** Difetti che impediscono l'esecuzione dei flussi principali, come crash del sistema, errori di avvio del backend o impossibilità di accedere alle pagine riservate.
- **Interruzioni dell'ambiente di test** Problemi legati all'ambiente di esecuzione, come malfunzionamenti del server locale, errori di configurazione, dipendenze mancanti o indisponibilità del database.
- **Incoerenza dei dati** Situazioni in cui il database presenta dati corrotti, duplicati o non coerenti con i flussi operativi, rendendo impossibile proseguire con test affidabili.
- **Impossibilità di completare un flusso operativo critico** Ad esempio: impossibilità di prenotare una visita, inserire un referto o effettuare il login.
- **Aggiornamenti del sistema in corso** Modifiche al codice, aggiornamenti delle dipendenze o interventi di manutenzione che richiedono la sospensione temporanea delle attività.
- **Condizioni esterne che impediscono la continuità del test** Interruzioni del dispositivo, problemi di rete o indisponibilità degli strumenti necessari.

Procedure di sospensione

Quando si verifica una condizione di sospensione:

1. **Registrare l'anomalia** nel documento di tracciamento dei difetti.
2. **Classificare il problema** indicando severità, priorità e modulo coinvolto.
3. **Documentare lo stato del test** (flusso interrotto, operazione non completata, dati non salvati).
4. **Informare il responsabile del testing** o il team di sviluppo.
5. **Interrompere tutte le attività correlate** al flusso interessato fino alla risoluzione del problema.

La sospensione deve essere sempre formalizzata per garantire tracciabilità e continuità del processo.

Condizioni di ripristino dei test

Le attività di testing possono essere ripristinate quando:

- il bug bloccante è stato risolto e verificato;
- l'ambiente di test è nuovamente stabile e funzionante;

- il database è stato ripristinato a uno stato coerente;
- le dipendenze software sono state correttamente reinstallate o aggiornate;
- il flusso operativo interrotto può essere eseguito senza errori;
- il sistema è nuovamente accessibile e funzionante in tutte le sue parti.

Procedure di ripristino

Per riprendere correttamente le attività di testing:

1. **Verificare la risoluzione del problema** tramite test mirati.
2. **Ripristinare lo stato del sistema** (database, configurazioni, sessioni).
3. **Ripetere il flusso operativo interrotto** per confermare la stabilità.
4. **Aggiornare la documentazione** indicando la data e l'ora del ripristino.
5. **Proseguire con i test pianificati** secondo la sequenza prevista nel Test Plan.

Risultati attesi

La gestione corretta della sospensione e del ripristino dei test deve garantire:

- continuità del processo di verifica;
- tracciabilità completa delle anomalie e delle risoluzioni;
- stabilità dell'ambiente di test;
- affidabilità dei risultati ottenuti;
- riduzione del rischio di errori nelle fasi successive.

8. Materiale e Ambiente di Testing

La presente sezione descrive in modo dettagliato il **materiale**, gli **strumenti** e l'**ambiente tecnico** utilizzati per eseguire le attività di testing del sistema **CardioCare360**. Una definizione chiara dell'ambiente di test è fondamentale per garantire la riproducibilità dei risultati, la coerenza delle verifiche e la corretta valutazione delle funzionalità implementate.

L'ambiente di testing è stato configurato in modo da rispecchiare fedelmente le condizioni operative previste per il sistema, assicurando che i test siano rappresentativi dell'esperienza reale degli utenti.

Materiale utilizzato

Il materiale impiegato durante le attività di testing comprende:

- **Computer portatile / Desktop** Utilizzato per eseguire backend, frontend e strumenti di verifica.
- **Browser Web (Chrome / Firefox / Edge)** Impiegato per testare l'interfaccia utente, la navigazione e i flussi operativi.
- **Editor di codice (VS Code / IntelliJ IDEA)** Utilizzato per eventuali verifiche del codice e per l'avvio del backend.
- **Terminale / Prompt dei comandi** Necessario per eseguire comandi di avvio, installazione dipendenze e analisi log.
- **LibreOffice Writer / Calc** Utilizzato per documentare i risultati dei test, registrare anomalie e compilare tabelle di tracciamento.
- **Postman** Impiegato per testare le API REST e verificare la correttezza delle risposte del backend.
- **MySQL Workbench** Utilizzato per visualizzare il database, eseguire query, verificare la coerenza dei dati e analizzare le tabelle generate dal backend.

Ambiente software

L'ambiente software utilizzato per il testing include:

- **Java 17** Versione utilizzata per eseguire il backend Spring Boot.
- **Spring Boot** Framework backend utilizzato per gestire API, logica di business e persistenza.
- **Node.js + npm** Necessari per l'esecuzione del frontend React e l'installazione delle dipendenze.
- **React** Framework frontend utilizzato per l'interfaccia utente.
- **MySQL + MySQL Workbench** Utilizzati per la gestione del database reale, la verifica della persistenza dei dati, l'esecuzione di query SQL e il controllo della struttura delle tabelle.
- **Database integrato in Spring Boot (H2 / configurazione locale)** Utilizzato nelle fasi preliminari di sviluppo e test rapido.
- **Maven** Gestore delle dipendenze backend.

- **Git** Utilizzato per versionamento e controllo del codice.

8.3 Ambiente hardware

L'ambiente hardware minimo richiesto per eseguire i test comprende:

- Processore dual-core o superiore.
- 8 GB di RAM (consigliati 16 GB per esecuzione simultanea di backend e frontend).
- Connessione Internet stabile (necessaria per installazione dipendenze e aggiornamenti).
- Spazio su disco sufficiente per repository, dipendenze e file di log.

Configurazione dell'ambiente di test

L'ambiente di test è stato configurato seguendo le specifiche del SDD:

- Installazione di Java, Node.js e npm.
- Configurazione del backend tramite Spring Boot.
- Installazione delle dipendenze frontend tramite npm.
- Configurazione del database integrato.
- Avvio simultaneo di backend e frontend.
- Verifica della comunicazione tra i due componenti.
- Controllo della corretta esposizione delle API tramite Postman.
- Verifica della corretta visualizzazione delle pagine principali.
- Configurazione del database MySQL.
- Verifica della connessione tra Spring Boot e MySQL.
- Controllo della creazione automatica delle tabelle tramite JPA/Hibernate.
- Analisi della persistenza dei dati tramite MySQL Workbench.

Obiettivi dell'ambiente di testing

L'ambiente di testing deve garantire:

- stabilità durante l'esecuzione dei test;
- coerenza dei risultati;
- riproducibilità delle condizioni operative;
- compatibilità con le specifiche del RAD e del SDD;
- affidabilità nella valutazione delle funzionalità principali.

Risultati attesi

L'ambiente di testing deve consentire l'esecuzione completa e corretta di tutte le attività previste dal Test Plan, garantendo che il sistema **CardioCare360** sia valutato in condizioni controllate, coerenti e rappresentative dell'utilizzo reale.

9. Test Cases

La presente sezione descrive i **casi di test** definiti per verificare il corretto funzionamento delle principali funzionalità del sistema **CardioCare360**. Ogni test case è progettato per validare i requisiti funzionali descritti nel **RAD** e nei **diagrammi di attività**, assicurando che il comportamento del sistema sia conforme alle specifiche.

9.1 Gestione Registrazione

9.1.1 Registrazione Paziente

Obiettivo: verificare la corretta creazione di un nuovo account paziente.

Input: dati anagrafici, email, password.

Risultato atteso: il sistema registra il paziente e mostra messaggio di conferma.

Criterio Pass/Fail: account creato e accessibile tramite login.

9.1.2 Registrazione Medico

Obiettivo: verificare la registrazione di un nuovo medico.

Input: dati professionali, email, password, specializzazione.

Risultato atteso: il sistema salva il profilo medico e abilita l'accesso.

Criterio Pass/Fail: medico registrato e visibile nel database.

9.2 Gestione Autenticazione

9.2.1 Login / Logout

Obiettivo: verificare la corretta autenticazione e disconnessione dell'utente.

Input: credenziali valide e non valide.

Risultato atteso: accesso consentito con credenziali corrette, negato con errate; logout invalida la sessione.

Criterio Pass/Fail: comportamento conforme alle regole di sicurezza.

9.3 Gestione Prenotazioni Visite

9.3.1 Prenotazione Visita Cardiologica

Obiettivo: verificare la corretta creazione di una prenotazione.

Input: tipo visita, data, orario, medico.

Risultato atteso: prenotazione salvata e visibile nella lista appuntamenti.

Criterio Pass/Fail: dati coerenti e conferma visualizzata.

9.3.2 Modifica / Annullamento Prenotazione

Obiettivo: verificare la possibilità di modificare o annullare una prenotazione esistente.

Input: nuova data/orario o richiesta di annullamento.

Risultato atteso: aggiornamento o cancellazione corretta nel database.

Criterio Pass/Fail: stato aggiornato e riflesso nella lista appuntamenti.

9.4 Gestione Esami e Referti

9.4.1 Inserimento Referto

Obiettivo: verificare che il medico possa inserire un referto associato a un esame.

Input: testo referto, note medico, identificativo esame.

Risultato atteso: referto salvato e visibile al paziente.

Criterio Pass/Fail: referto correttamente associato e recuperabile.

9.4.2 Visualizzazione Esame

Obiettivo: verificare che il paziente possa visualizzare i propri esami.

Input: ID paziente.

Risultato atteso: elenco esami con dettagli e referti.

Criterio Pass/Fail: dati corretti e accesso consentito solo al paziente.

9.5 Gestione Terapie

9.5.1 Aggiunta / Aggiornamento Terapia

Obiettivo: verificare la gestione delle terapie da parte del medico.

Input: nome farmaco, dosaggio, durata, note.

Risultato atteso: terapia salvata o aggiornata correttamente.

Criterio Pass/Fail: dati coerenti e visibili nel profilo paziente.

9.6 Gestione Profilo Utente

9.6.1 Modifica Dati Personali

Obiettivo: verificare la possibilità di aggiornare i dati personali dell'utente.

Input: nuovi dati anagrafici o di contatto.

Risultato atteso: aggiornamento salvato e confermato.

Criterio Pass/Fail: dati aggiornati e correttamente visualizzati.

9.6.2 Visualizzazione Storico

Obiettivo: verificare la corretta visualizzazione dello storico visite ed esami.

Input: ID utente.

Risultato atteso: elenco completo e ordinato di visite, esami e terapie.

Criterio Pass/Fail: dati coerenti con il database e correttamente filtrati.